MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

María Sonia Salido Fernández

Módulo 1 - Tarea 2

Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- **1. Entendiendo qué pide el ejercicio**
- **2. Extraer de los PEM los parámetros DH**
- **3. Validar criptográficamente las claves públicas**
  - **3.1 ¿Cumplen ambas claves públicas con lo mínimo exigido?**
  - **3.2 ¿Son compatibles las claves públicas entre sí? ¿Pertenecen al mismo grupo?**
  - **3.3 ¿El grupo elegido es seguro?**
    - **Factorizando p-1**
- **4. Recuperar una clave privada mediante Pohlig Hellman**
- **5. Calcular el secreto compartido DH**
- **6. Derivar la clave AES con HKDF-SHA256**
- **7. Tomar los primeros 16 bytes de gon.enc como IV & Descifrar el resto con AES-CBC**
- **8. Conclusiones**

Entendiendo qué pide el ejercicio

Partes para resolver:

- Recuperar la clave AES.
- Usar esta clave para descifrar `gon.enc`.

Según el enunciado, el ransomware acuerda un secreto con Diffie-Hellman, lo pasa por HKDF-SHA256 con info = "shared key" y después cifra los archivos con AES-CBC. Además, el IV está en los primeros 16 bytes del fichero cifrado.

El punto decisivo:

- AES-CBC sí se puede revertir si tenemos la clave y el `IV`.
- El `IV` ya lo tenemos: está guardado al principio de `gon.enc`.
- La clave AES no está guardada: se deriva del secreto DH.
- Por tanto, todo dependerá de si podemos reconstruir el secreto Diffie-Hellman sólo a partir de las claves públicas.

Sabemos que tenemos:

- `dhPubCC.pem`: clave pública del servidor C&C.
- `dhPubLocal.pem`: clave pública de la máquina víctima.
- `gon.enc`: un archivo cifrado cualquiera.
- El IV está en los primeros 16 bytes de `gon.enc`.
- La clave privada local de DH fue borrada y no queda rastro de la clave AES ni del original.

El objetivo es reconstruir:

- El secreto compartido DH.
- La clave derivada de 32 bytes con HKDF.
- Usar esa clave con AES-CBC y el `IV` para descifrar.

Entendiendo qué operación produce la clave de cifrado:
Esto es lo que el ransomware hace:

- Calcula el secreto compartido DH: $s=(Y_{CC})^{xlocal} \bmod (p)$.
    - Y_{CC} es la clave pública del servidor.
    - xlocal es la clave privada local borrada.
- Deriva: K=HKDF-SHA256(s, salt=∅, info="shared key", len=32)
- Usa `k` como clave AES-CBC para todos los ficheros.
- Conclusión: Tenemos que recuperar `s` para recuperar `k`.

2. Extraer de los PEM los parámetros DH

El enunciado del ejercicio dice que esas dos claves públicas son la única información criptográfica que quedó en el disco. Vamos a abrir los ficheros `dhPubCC.pem` y `dhPubLocal.pem` para extraer los parámetros de Diffie-Hellman. También tendremos comprobar que:

- Que ambas claves usan los mismos parámetros `p` y `g`.
- Que los valores públicos tienen forma válida para ese grupo.

Vamos a extraer de cada PEM: De cada clave pública DH nos interesan estos valores:

- `p`: El primo del grupo.
- `g`: El generador.
- `y`: El valor público de esa parte:
  - Para `dhPubLocal.pem`, sería el `Y_local`.
  - Para `dhPubCC.pem` sería `Y_CC`

Luego haremos la comparación:

- Si `p` es el mismo en ambos.
- Si `g` es el mismo en ambos.

Si NO coinciden, NO pertenecen al mismo intercambio DH.

Inspeccionamos la clave pública local:

```
└─$ openssl pkey -pubin -in dhpublocal.pem -text -noout
DH Public-Key: (1024 bit)
public-key:
    06:e8:c8:d4:19:5a:6d:2d:6f:7e:c5:9c:2f:bc:71:
    d4:7e:ed:c3:28:ae:89:20:78:36:8d:e2:59:26:f6:
    6f:7f:f6:14:0e:17:21:0a:f8:31:73:64:1f:52:2d:
    6d:92:cb:78:c6:a2:dd:d0:0a:99:2d:6f:7e:9c:65:
    a4:44:a7:3d:0f:b6:cb:b5:28:08:90:02:6c:0e:7e:
    9a:e2:83:9f:00:6b:30:02:ef:f3:1a:4f:8f:4e:8a:
    d8:32:1d:81:be:a8:f6:d0:a2:aa:b2:00:3a:fc:b0:
    bb:29:74:5d:1c:31:b1:96:0f:0d:4b:de:ea:de:e8:
    25:fe:d0:c3:f4:fd:1d:f5
P:
    00:ef:04:06:4e:89:29:35:71:2a:d8:82:11:ba:6c:
    1a:9e:58:a8:45:c7:dc:3e:b5:3d:13:95:7c:20:0e:
    94:6a:f4:94:54:76:7f:fd:45:d1:4a:74:2c:84:09:
    27:df:2e:d7:6c:83:dc:50:f5:ac:2b:45:8d:6b:ed:
    32:a7:a1:bf:4a:12:29:78:38:c5:eb:54:21:e8:f5:
    a7:c5:53:77:12:69:d2:54:4e:29:8b:d1:2f:10:47:
    40:e7:52:13:45:f1:73:2e:f3:f4:d6:b7:d6:fe:94:
    33:85:c9:74:54:d4:68:6d:94:6b:38:80:7a:b3:d8:
    b4:3a:0d:e5:2f:df:69:57:17
```

```
G:    5 (0x5)
```

donde:

- `public-key` → Y_{local} → `06e8c8d4...`.
- `prime` → `p` → `00ef0406...`.
- `generator` → `g` → `5`.

Inspeccionamos la clave pública del C&C:

```
└─$ openssl pkey -pubin -in dhpubCC.pem -text -noout
DH Public-Key: (1024 bit)
public-key:
    1d:12:bb:62:34:4d:ac:6d:b5:e1:0c:9d:f4:45:72:
    64:62:65:df:ce:9c:7d:59:a9:a0:2a:7d:88:09:6d:
    64:43:47:39:58:6f:e3:2a:c8:bb:31:5e:a1:f3:2e:
    5c:1a:0a:4a:ff:db:5c:13:ae:f5:3d:07:44:48:ae:
    67:56:8c:a0:3a:69:96:30:34:97:38:61:18:b3:1c:
    1d:1c:29:2a:72:fd:44:eb:7f:f4:34:1d:49:6b:f0:
    11:bf:5a:5b:e0:aa:9b:1a:68:4d:00:d7:9c:e4:e4:
    da:a7:50:47:79:f2:90:7f:bf:3f:e2:02:73:e0:b0:
    80:a7:c5:10:f6:71:9c:aa
P:
    00:ef:04:06:4e:89:29:35:71:2a:d8:82:11:ba:6c:
    1a:9e:58:a8:45:c7:dc:3e:b5:3d:13:95:7c:20:0e:
    94:6a:f4:94:54:76:7f:fd:45:d1:4a:74:2c:84:09:
    27:df:2e:d7:6c:83:dc:50:f5:ac:2b:45:8d:6b:ed:
    32:a7:a1:bf:4a:12:29:78:38:c5:eb:54:21:e8:f5:
    a7:c5:53:77:12:69:d2:54:4e:29:8b:d1:2f:10:47:
    40:e7:52:13:45:f1:73:2e:f3:f4:d6:b7:d6:fe:94:
    33:85:c9:74:54:d4:68:6d:94:6b:38:80:7a:b3:d8:
    b4:3a:0d:e5:2f:df:69:57:17
G:    5 (0x5)
```

donde:

- `public-key` → Y_{CC} → `1d12bb62...`.
- `prime` → `p` → `00ef0406...`.
- `generator` → `g` → `5`

Conclusión: Ambas claves públicas DH usan los mismos parámetros de dominio:

- Mismo primo `p` = 00ef0406...
- Mismo generador `g`=5.
- **Por tanto, `dhPubLocal.pem` y `dhPubCC.pem` pertenecen al mismo grupo Diffie-Hellman, y sí pueden formar parte del mismo intercambio DH. Eso corrobora el escenario del enunciado del ejercicio.**

Notas sobre el primo:

- El 00 inicial del primo no cambia el valor matemático del primo. Suele aparecer en la codificación `ASN.1/DER` para indicar que el entero es positivo. Entonce el valor real del primo es:

```
ef04064e892935712ad88211ba6c1a9e58a845c7dc3eb53d13957c200e946af49454767ffd4
5d14a742c840927df2ed76c83dc50f5ac2b458d6bed32a7a1bf4a12297838c5eb5421e8f5a7
c553771269d2544e298bd12f104740e7521345f1732ef3f4d6b7d6fe943385c97454d4686d9
46b38807ab3d8b43a0de52fdf695717
```

- OpenSSL indica 1024 bits. Eso describe el tamaño de `p`.

3. Validar criptográficamente las claves públicas

Vamos a aplicar la pista: Vamos a validar las claves públicas. Con Diffie-Hellman, no basta con ver una clave pública y asumir que es normal. Hay que comprobar si:

- Está en el rango correcto.
- Pertenece al subgrupo esperado.
- No es un valor degenerado como 1, p-1, 0, etc.
- No fuerza un secreto compartido trivial o de muy pocos valores.

3.1 ¿Cumplen ambas claves públicas con lo mínimo exigido?

- $1 < Y_{local} < p-1$.
- $1 < Y_{CC} < p-1$.

Esto implica que no estamos ante casos triviales como `0`, `1` o `p-1`.

3.2 ¿Son compatibles las claves públicas entre sí? ¿Pertenecen al mismo grupo?

- Ambas usan el mismo `p`.
- Ambas usan el mismo `g`.

Esto implica que son compatibles entre sí. Las dos claves públicas están definidas sobre el mismo dominio DH. Si una usara otro `p` u otro `g`, ni siquiera tendría sentido hablar del mismo intercambio.

3.3 ¿El grupo elegido es seguro?

Para saber si el grupo elegido es seguro, debemos mirar la estructura de grupo, es decir, debemos estudiar cómo está compuesto `p-1`.

Para evaluar la seguridad del grupo DH hay que estudiar la factorización de `p−1`, ya que el orden del grupo multiplicativo módulo `p` es precisamente `p−1`. Si ese orden tiene un factor primo grande, el grupo puede ser adecuado para `Diffie–Hellman`. Si, por el contrario, `p−1` es muy suave y se descompone en factores pequeños, el logaritmo discreto se simplifica mediante `Pohlig–Hellman`, debilitando el intercambio.

Factorizando p-1

Script python para factorizar p-1:

```
from sympy import factorint

p =
int("ef04064e892935712ad88211ba6c1a9e58a845c7dc3eb53d13957c200e946af4945476
7ffd45d14a742c840927df2ed76c83dc50f5ac2b458d6bed32a7a1bf4a12297838c5eb5421e
8f5a7c553771269d2544e298bd12f104740e7521345f1732ef3f4d6b7d6fe943385c97454d4
686d946b38807ab3d8b43a0de52fdf695717", 16)

f = factorint(p - 1)
print(f)
```

Salida:

```
└─$ python3 factorizar.py
{2: 1, 5786257: 9, 7166147: 11, 5015911: 4, 4049039: 10, 3672281: 12}
```

donde:

- Se obtiene: $p-1 = 2 \cdot 5786257^{9} \cdot 7166147^{11} \cdot 5015911^{4} \cdot 4049039^{10} \cdot 3672281^{12}$
- Vemos que `p−1` está completamente factorizado.
- Sus factores primos son relativamente pequeños.

Conclusión: Se verificó que ambas claves públicas DH están en el rango válido y usan los mismos parámetros `(p,g)`. Sin embargo, la seguridad del intercambio no depende sólo de eso. Al analizar el primo, se obtuvo que `p−1` factoriza completamente en potencias de factores relativamente pequeños, por lo que el grupo multiplicativo módulo `p` es suave. En estas condiciones, el logaritmo discreto puede resolverse mediante `Pohlig–Hellman`, recuperando la clave privada a partir de la pública. Por tanto, el secreto compartido DH no está realmente protegido y sí puede reconstruirse, lo que permitiría derivar la clave AES y descifrar el fichero..

4. Recuperar una clave privada mediante Pohlig Hellman

Una vez establecida la debilidad del grupo, el siguiente paso es resolver el logaritmo discreto de una de las claves públicas para recuperar una clave privada DH mediante `Pohlig-Hellman`. Con esa clave privada y la clave pública de la otra parte se reconstruye el secreto compartido. A continuación, se deriva la clave de 32 bytes con `HKDF-SHA256`, sin `salt` y con `info = "shared key"`. Finalmente, se toma el `IV` de los primeros 16 bytes del fichero cifrado y se descifra el resto con `AES-CBC`.

A partir de la clave pública `Diffie-Hellman` se tiene: $Y_{local}=g^{x_{local}} \bmod (p)$.

donde:

- `p` es el primo del grupo.
- `g` es el generador.
- Y_{local} es la clave pública local.
- x_{local} es la clave privada que queremos recuperar.

Para recuperar la clave privada tenemos que resolver el siguiente logaritmo discreto: $x_{local} = \log_g(Ylocal) \pmod{ord(g)}$

Como se ha comprobado que `p-1` factoriza completamente en factores pequeños, este logaritmo discreto puede resolverse mediante el algoritmo de `Pohlig-Hellman`. Así que vamos a utilizar el **algoritmo de Pohlig-Hellman en SageMath** para resolver el problema del logaritmo discreto y averiguar la clave privada local o la del C&C. Para ello vamos a **seguir el código que aparece en la Práctica 4.16** donde debemos introducir el primo `p`, el generador `g` y utilizar una de las claves públicas extraídas como la variable `h` para que el algoritmo nos devuelva el valor de la clave privada `x`.

Script de python:

```
from sage.all import *

p =
0xef04064e892935712ad88211ba6c1a9e58a845c7dc3eb53d13957c200e946af49454767ff
d45d14a742c840927df2ed76c83dc50f5ac2b458d6bed32a7a1bf4a12297838c5eb5421e8f5
a7c553771269d2544e298bd12f104740e7521345f1732ef3f4d6b7d6fe943385c97454d4686
d946b38807ab3d8b43a0de52fdf695717
g = 5
h =
0x1d12bb62344dac6db5e10c9df44572646265dfce9c7d59a9a02a7d88096d64434739586fe
32ac8bb315ea1f32e5c1a0a4affdb5c13aef53d074448ae67568ca03a6996303497386118b3
1c1d1c292a72fd44eb7ff4341d496bf011bf5a5be0aa9b1a684d00d79ce4e4daa7504779f29
07fbf3fe20273e0b080a7c510f6719caa

print("p =", p)

F = GF(p)
g1 = F(g)
h1 = F(h)
```

```
N = p - 1
fac = factor(N)
print("Factorización de p-1 =", fac)

mods = []
residues = []

for q, e in fac:
    qe = q**e
    Ni = N // qe
    gi = g1**Ni
    hi = h1**Ni
    xi = discrete_log(hi, gi)
    mods.append(Integer(qe))
    residues.append(Integer(xi))
    print(f"x mod {qe} = {xi}")

x = crt(residues, mods)

print("x =", x)
print("Verificación =", power_mod(g, x, p) == h)
```

Instalación de `conda`: Para utilizar sageMath necesitamos previamente instalar `conda`:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
source ~/.bashrc
conda --version
```

donde:

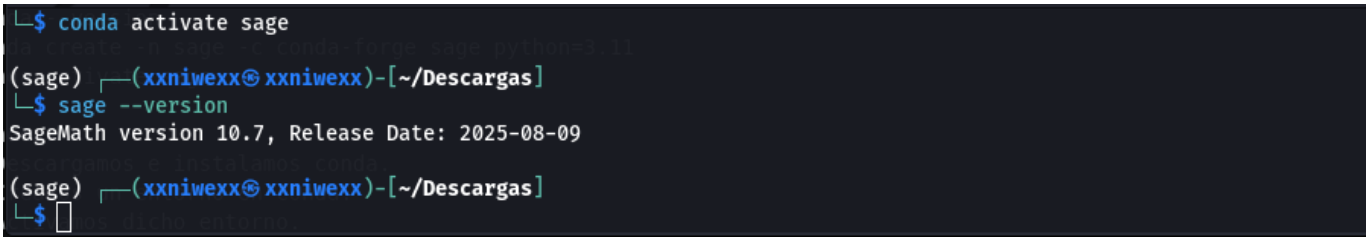
- Descargamos e instalamos miniconda.
- Se recarga la configuración del entorno de shell
- Verificamos su instalación mostrando la version de `conda`.

Se añade el canal `conda-forge` y se crea un entorno con SageMath:

```
conda config --add channels conda-forge
conda config --set channel_priority strict
conda create -n sage sage python=3.12
```

Activamos el entorno y comprobamos que funciona:

```
conda activate sage
sage --version
```



Iniciamos sageMath y ejecutamos el script:

```
sage -python sageMath.py
```

Obtenemos:

```
xxniwexx@xxniwexx: ~/Escritorio/cybersecurity/Documentation/Malware/Master-ENIIT-Analisis-Malware-Re...

(base) ┌──(xxniwexx㉿xxniwexx)-[~/…/Malware/Master-ENIIT-Analisis-Malware-Reversing/modulo-1-criptografia/02-M1T2-DHKE]
└─$ conda activate sage

(sage) ┌──(xxniwexx㉿xxniwexx)-[~/…/Malware/Master-ENIIT-Analisis-Malware-Reversing/modulo-1-criptografia/02-M1T2-DHKE]
└─$ sage --version
SageMath version 10.7, Release Date: 2025-08-09

(sage) ┌──(xxniwexx㉿xxniwexx)-[~/…/Malware/Master-ENIIT-Analisis-Malware-Reversing/modulo-1-criptografia/02-M1T2-DHKE]
└─$ sage -python sageMath.py
p = 16784254733836904465188327710940654881926675019144567885315230766121782345873672932113091401424612297302492395065408737548
5914651728186759220217638936253635128828183646608497089097820873088929013877944441359476095046643381870366547903562731352965 48
2816977854632506272468458312348751026325427705083023090865943
Factorización de p-1 = 2 * 3672281^12 * 4049039^10 * 5015911^4 * 5786257^9 * 7166147^11
x mod 2 = 1
x mod 601493402319161950053571503743667132543743886224903459188009177789038265660176 1 = 1119057941953991793429233995044258886 9
4752851561062573071732317813089373591565 6
x mod 1184457817785631815534960114722554474624465666166706127753882101601 = 6867449051636522728304907450948416185753372108743 1
2334950398878411
x mod 63299355461279000622072624 1 = 8785366778687878335386647 2
x mod 7270754364408433363204760201541280895527743160232012802179857 = 1405233454962415414426975869916971930248753566378704859 2
65549
x mod 2559427542685419980947509375768444576048523943668989850194811550737128390203 = 7431624700161717352179326016210501922119 1
2871852847391816745733491582366846
x = 7297838435444891171979861782378651046426208518086224080768431969336257741295937252537765340907391953292003644273514885126 6
9788242105977372426933870781562852782415711251590426380132154208847569485918948352520165275130032319897087867117863636588000418610041237006399992566462192881577434488318345321020 31613969
Verificación = True

(sage) ┌──(xxniwexx㉿xxniwexx)-[~/…/Malware/Master-ENIIT-Analisis-Malware-Reversing/modulo-1-criptografia/02-M1T2-DHKE]
└─$
```

Resultado obtenido: Para la clave pública del C&C: $Y_{CC}=g^{x_{CC}} \bmod (p)$.

```
x =
7297838435444891171979861782378651046426208518086224080768431969336257741295937252537765340907391953292003644273514885126697882421059773724269338707815628527824157112515904263801321542088475694859189483525201652751300323198970787671178636365880004186100412370063999925664621928815774344883183453210203161396 9
Verificación = True
```

x_{CC}=7297838435444891...

Se comprueba que: $5^{x_{CC}} \bmod (p) = Y_{CC}$

Con ello queda recuperada una de las claves privadas DH y ya puede reconstruirse el secreto compartido.

5. Calcular el secreto compartido DH

Ahora que ya hemos recuperado clave la privada del C&C, debemos combinarla con la clave pública local. Para ello vamos a calcular:

$s=Y_{local}^{x_{CC}} \bmod (p)$.

donde:

- Y_{local} es la clave pública de `dhPubLocal.pem`.
- x_{CC} es la clave privada que acabamos de recuperar.
- p es el primo del grupo.

Ese valor `s` es el `shared secret` que usó el ransomware antes de derivar la clave simétrica.

En los apuntes, para DH se define que Alice calcula el secreto compartido como $k_s = B^a \bmod(p)$, y Bob como $k_s = A^b \bmod(p)$. Ambos resultados son idénticos y equivalen a $g^{ab} \bmod(p)$. Donde:

- La clave pública local hace el papel de A.
- La clave pública del C&C hace el papel de B.
- La clave privada del C&C recuperada hace el papel de b.

Por tanto, el secreto compartido se expresa como: $k_s = Y_{local}^{x_{CC}} \bmod(p)$.

La fórmula equivalente desde el otro lado: $k_s = Y_{CC}^{x_{local}} (\bmod p)$.

donde:

- Y_{local} es la clave pública local.
- x_{CC} es la clave privada del servidor C&C.

Ambas expresiones producen el mismo secreto compartido, exactamente igual que en el tema del módulo con $B^a \bmod(p)$ y $A^b \bmod(p)$.

Y_{local} es:

```
06e8c8d4195a6d2d6f7ec59c2fbc71d47eedc328ae892078368de25926f66f7ff6140e17210
af83173641f522d6d92cb78c6a2ddd00a992d6f7e9c65a444a73d0fb6cbb5280890026c0e7e
9ae2839f006b3002eff31a4f8f4e8ad8321d81bea8f6d0a2aab2003afcb0bb29745d1c31b19
60f0d4bdeeadee825fed0c3f4fd1df5
```

x_{CC} es:

```
72978384354448911719798617823786510464262085180862240807684319693362577412959372525377653409073919532920036442735148851266978824210597737242693387078156285278241571125159042638013215420884756948591894835252016527513003231989708786711786363658800041861004123700639999256646219288157743448831834532102031613969
```

p es:

```
ef04064e892935712ad88211ba6c1a9e58a845c7dc3eb53d13957c200e946af49454767ffd45d14a742c840927df2ed76c83dc50f5ac2b458d6bed32a7a1bf4a12297838c5eb5421e8f5a7c553771269d2544e298bd12f104740e7521345f1732ef3f4d6b7d6fe943385c97454d4686d946b38807ab3d8b43a0de52fdf695717
```

Sustituyendo los valores obtenidos:

k_S=(06e8c8d4…f4fd1df5)$^{x_{CC}}$mod(ef04064e…df695717)

Y se obtiene el secreto compartido:

```
14161774707882020190207306489911486199691017207255571187769877449033916452383424270585365492008620293100173132411704682282330843018345165420852837434004216753945212139612494054015238759776091975456507093984549228512926361714667103477132500221761177163636815929314888839806653196831446516523776568011307778483 7
```

Que en hexadecimal es:

```
c9ab9d1bb9e613e08d7a4768a28af685d0f634d684638cfa04a58dfce769cf4bad12e53407b4e3ede7c867f101b66a6f4f316a40895b2d4294e5e055990c09341daa7c6a5820a3a7409e89d27630f31c7e774ab0ccb047bfc0e217a7b5dddc709ac1a3cfb4c731094fb88ae9e2170417b7efc530432b16f8939539d205491105
```

Donde:

- **Obtenemos que el secreto compartido en hexadecimal es `c9ab9d1bb9e613e08d7a...`. Corresponde a 128 bytes, coherentes con un grupo DH de 1024 bits.**

Otra forma de calcular el secreto compartido: Sin usar sageMath podemos usar un script de python:

Script Python para calcular el secreto compartido:

```
p =
int("ef04064e892935712ad88211ba6c1a9e58a845c7dc3eb53d13957c200e946af4945476
7ffd45d14a742c840927df2ed76c83dc50f5ac2b458d6bed32a7a1bf4a12297838c5eb5421e
8f5a7c553771269d2544e298bd12f104740e7521345f1732ef3f4d6b7d6fe943385c97454d4
686d946b38807ab3d8b43a0de52fdf695717", 16)

Y_local =
int("06e8c8d4195a6d2d6f7ec59c2fbc71d47eedc328ae892078368de25926f66f7ff6140e
17210af83173641f522d6d92cb78c6a2ddd00a992d6f7e9c65a444a73d0fb6cbb5280890026
c0e7e9ae2839f006b3002eff31a4f8f4e8ad8321d81bea8f6d0a2aab2003afcb0bb29745d1c
31b1960f0d4bdeeadee825fed0c3f4fd1df5", 16)

x_cc =
729783843544489117197986178237865104642620851808622408076843196933625774129
593725253776534090739195329200364427351488512669788242105977372426933870781
562852782415711251590426380132154208847569485918948352520165275130032319897
087867117863636588000418610041237006399992566462192881577434488318345321020
31613969

s = pow(Y_local, x_cc, p)

print("s =", s)
print("hex(s) =", hex(s)[2:])
```

Obtenemos:

```
└─$ python3 calcular_secreto.py
s =
141617747078820201902073064899114861996910172072555711877698774490339164523
834242705853654920086202931001731324117046822823308430183451654208528374340
042167539452121396124940540152387597760919754565070939845492285129263617146
671034771325002217611771636368159293148888398066531968314465165237765680113
077784837
hex(s) =
c9ab9d1bb9e613e08d7a4768a28af685d0f634d684638cfa04a58dfce769cf4bad12e53407b
4e3ede7c867f101b66a6f4f316a40895b2d4294e5e055990c09341daa7c6a5820a3a7409e89
d27630f31c7e774ab0ccb047bfc0e217a7b5dddc709ac1a3cfb4c731094fb88ae9e2170417b
7efc530432b16f8939539d205491105
```

donde:

- **El compartido en hexadecimal es:** `s = c9ab9d1bb9e613e...`.
- Ocupa 128 bytes, que encaja con un grupo DH de 1024 bits.

Convertimos el secreto compartido `s` a bytes:

```
s_hex =
"c9ab9d1bb9e613e08d7a4768a28af685d0f634d684638cfa04a58dfce769cf4bad12e53407
b4e3ede7c867f101b66a6f4f316a40895b2d4294e5e055990c09341daa7c6a5820a3a7409e8
9d27630f31c7e774ab0ccb047bfc0e217a7b5dddc709ac1a3cfb4c731094fb88ae9e2170417
b7efc530432b16f8939539d205491105"
s_bytes = bytes.fromhex(s_hex)

print("s_bytes =", s_bytes)
print("longitud =", len(s_bytes), "bytes")
```

Obtenemos:

```
└─$ python3 convertir_s_en_bytes.py
s_bytes =
b'\xc9\xab\x9d\x1b\xb9\xe6\x13\xe0\x8dzGh\xa2\x8a\xf6\x85\xd0\xf64\xd6\x84c
\x8c\xfa\x04\xa5\x8d\xfc\xe7i\xcfK\xad\x12\xe54\x07\xb4\xe3\xed\xe7\xc8g\xf
1\x01\xb6joO1j@\x89[-B\x94\xe5\xe0U\x99\x0c\t4\x1d\xaa|jX
\xa3\xa7@\x9e\x89\xd2v0\xf3\x1c~wJ\xb0\xcc\xb0G\xbf\xc0\xe2\x17\xa7\xb5\xdd
\xdcp\x9a\xc1\xa3\xcf\xb4\xc71\tO\xb8\x8a\xe9\xe2\x17\x04\x17\xb7\xef\xc50C
+\x16\xf8\x93\x959\xd2\x05I\x11\x05'
longitud = 128 bytes
```

donde:

- **Una vez calculado el secreto compartido `s`, se convierte a su representación binaria para poder usarlo como entrada de HKDF. En este caso, el valor ocupa 128 bytes, lo que concuerda con el tamaño del módulo `Diffie-Hellman` de 1024 bits.**

6. Derivar la clave AES con HKDF-SHA256

Una vez reconstruido el secreto compartido K_S mediante `Diffie-Hellman`, este valor no debe emplearse directamente como clave secreta. El propio tema del módulo indica que, en la práctica, el secreto compartido de DH se utiliza como semilla para una función de derivación de claves (KDF), ya que su representación binaria no tiene por qué ser uniformemente aleatoria ni adecuada para usarse directamente como clave simétrica.

Además, el tema también señala que el secreto compartido puede emplearse para derivar una clave de sesión para un algoritmo simétrico como AES.

El enunciado del ejercicio dice exactamente que el malware:

- deriva una clave de 32 bytes,
- usando HKDF con SHA-256,
- sin salt,
- con info = "shared key"
- Conclusión:
    - K=HKDF-SHA256(s, salt=∅, info="shared key", 32)
    - Ese `K` es la clave que luego se usará en AES-CBC.

El objetivo ahora es derivar la clave AES de 32 bytes a partir del `s_bytes` usando exactamente:

- HKDF
- SHA-256
- salt = None
- info = b"shared key"
- length = 32

Para solucionar esa ecuación he buscado un script para python que haga la derivación:

```
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes

s_hex =
"c9ab9d1bb9e613e08d7a4768a28af685d0f634d684638cfa04a58dfce769cf4bad12e53407
b4e3ede7c867f101b66a6f4f316a40895b2d4294e5e055990c09341daa7c6a5820a3a7409e8
9d27630f31c7e774ab0ccb047bfc0e217a7b5dddc709ac1a3cfb4c731094fb88ae9e2170417
b7efc530432b16f8939539d205491105"
s_bytes = bytes.fromhex(s_hex)

hkdf = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=None,
    info=b"shared key",
)
```

```
key = hkdf.derive(s_bytes)

print("key =", key)
print("key_hex =", key.hex())
print("longitud =", len(key), "bytes")
```

Obtenemos:

```
└─$ python3 derivacion.py
key =
b'TR\x1b\xb0\xab\xfe^3n|\x80\xc3=\xe7\x00l\xcc\xe7\xf8\xf3\xdf\xa9\x1b3<+Q\
xab\x95;\x13\x85'
key_hex = 54521bb0abfe5e336e7c80c33de7006ccce7f8f3dfa91b333c2b51ab953b1385
longitud = 32 bytes
```

donde:

- `key`: Es la clave derivada por HKDF-SHA256.
- `key_hex` es esa misma clave, pero escrita como texto hexadecimal para poder verla, copiarla y usarla fácilmente
- **La cadena `54521bb0abfe5e336e7c80c33de7006ccce7f8f3dfa91b333c2b51ab953b1385` es la clave AES-256 que usó el malware para cifrar los archivos, porque el enunciado dice que la clave derivada es de 32 bytes y luego se usa en AES-CBC.**

7. Tomar los primeros 16 bytes de gon.enc como IV & Descifrar el resto con AES-CBC

Ahora vamos a descifrar `gon.enc` con AES-CBC usando:

- La clave AES: 54521bb0abfe5e336e7c80c33de7006ccce7f8f3dfa91b333c2b51ab953b1385.
- El IV: los primeros 16 bytes de `gon.enc`.
- El `ciphertext`: el resto del archivo.

Extraemos el `IV` de los primeros 16 bytes:

```
└─$ IV=$(xxd -p -l 16 gon.enc | tr -d '\n')
echo "$IV"
fb06da24b4f15e9c5e48bb7e5424d6d0
```

donde:

- Obtenemos el `IV` en hexadecimal: `fb06da24b4f15e9c5e48bb7e5424d6d0`.

Extraemos el ciphertext:

```
tail -c +17 gon.enc > gon.ct
```

donde:

- Se genera un fichero `gon.ct` que contiene únicamente el texto cifrado, eliminando los primeros 16 bytes de `gon.enc`, ya que esos bytes corresponden al vector de inicialización `IV` y no forman parte del bloque cifrado. De este modo, `gon.ct` queda preparado para ser descifrado con AES-CBC usando por separado la clave derivada y el `IV` extraído del encabezado del fichero.

Descifrar con AES-256-CBC: Como ya téniamos la clave derivada: `54521bb0abfe5e336e7c80c33de7006ccce7f8f3dfa91b333c2b51ab953b1385`, ahora desciframos con openssl:

```
└─$ openssl aes-256-cbc -d \
  -in gon.ct \
  -out gon.dec \
  -K 54521bb0abfe5e336e7c80c33de7006ccce7f8f3dfa91b333c2b51ab953b1385 \
  -iv fb06da24b4f15e9c5e48bb7e5424d6d0

└─$ cat gon.dec
La criptografía es como un juego de ajedrez, solo que los peones y las
reinas son números y letras.
```

donde:

- **El fichero descifrado contiene el mensaje: `La criptografía es como un juego de ajedrez, solo que los peones y las reinas son números y letras.`, lo que verifica que la clave derivada y el proceso de descifrado son correctos.**

8. Conclusiones

Sí, `gon.enc` puede descifrarse. Aunque la clave privada local fue eliminada, el intercambio `Diffie-Hellman` se realizó sobre un grupo inseguro, ya que `p-1` factoriza en potencias de factores pequeños. Esto permite aplicar `Pohlig-Hellman` para resolver el logaritmo discreto y recuperar una clave privada a partir de una clave pública. Con esa clave privada se reconstruye el secreto compartido DH, se deriva la clave simétrica de 32 bytes mediante `HKDF-SHA256`, sin `salt` y con `info = "shared key"`, y finalmente se descifra el fichero usando `AES-CBC` con el `IV` almacenado en sus primeros 16 bytes.